

Sorting Cont'd

1.Upper bound and Lower bound

1.1 Complexity of a problem

- 复杂度是什么?
- The complexity of the best algorithms that allow solving the problem.

1.2 Upper bound and Lower bound

- **Upper bound:**The worst runtime of an algorithm A on input of size n is:
$$T_A(n) = \max_{|I|=n} \{cost_A(I)\}$$
- **Lower bound:**The worst-case complexity of P is the worst-case runtime of the fastest algorithm that solves : $T_P(n) = \min_{A \text{ solves } P} \{ \max_{|I|=n} \{cost_A(I)\} \}$

2.Adversary Argument

2.1 Key operation

- A key operation is a step that is representative of the computation overall.
- 键操作的属性:
 - 时间固定
 - 代表或控制其他操作
 - 键操作的数量应该给输入的大小一个函数关系
- 一个计算模型将会在键操作旁建立（其他的计算会被忽略）

2.2 An Example:Three-Card Monte

2.3 Adversary Argument

一个全能的恶意对手假装为算法选择一个输入。当算法想要检查一个比特时，攻击者将该比特设置为任何能使算法完成最多工作的值。如果算法在终止前没有检查足够的比特，那么将会有几个不同的输入，每个输入与已经检查的比特一致，并且应该导致不同的输出。无论算法的输出是什么，攻击者都可以“揭示”一个包含所有已检查比特但与算法输出相矛盾的输入，然后声称这是他一直在使用的输入。

这个论点的一个绝对关键的特征是对手完全不对算法做任何假设。对手的策略不能依赖于一些预先确定的检查位的顺序，它也不关心算法在不看位的时候可能做什么或者不做什么。然而，只要问题至少有两个可能的答案与对手给出的所有答案一致，算法就无法完成。

2.4 Adversary argument for Comparison-based Sorting

- 比较作为一个 key operation 广泛应用在排序算法的分析中。

3.Information-Theoretic Arguments

3.1 The amount of the information

- M 是一个排序算法所有不同的输出中最小的总输出, 输入的数组长度为 n , 则 $M=n!$
- 如果一个确定的算法作了 c 次比较, 会产生多少种不同的输出?
- 最多 2^c 种不同的输出
- 因此, $2^c \geq n!$ 所以 c 最小为 $\lg n!$

3.2 Information content

- **Shannon information:**
- $I(x) = -\lg Pr(x)$

3.3 Information entropy

给出一个离散型随机变量 X , 该变量取字母 α 中的值, 并根据其分布, 随机变量的熵是该变量可能结果固有的“信息”、“惊喜”或“不确定性”的平均水平。

$$H(X) = \sum_{x \in \alpha} -Pr(X = x) \lg Pr(X = x) = E[-\lg Pr(X)]$$

4.Non-comparison-based Sorting

4.1 BucketSort

- 创建 d 个空的链表 (也就是桶)
- 遍历输入的数据, 对于每个元素, 把它添加到相对应链表的尾端
- 把所有链表合并

时间复杂度为 $\Theta(n)$, 稳定

```
1 BucketSort(A,k):
2   <L_1,L_2,...,L_k> = CreateBucket(k)
3   for i:=1 to A.length
4       AssignToBucket(A[i])
5   for j:=1 to k
6       SortWithinBucket(L_j)
7   CombineBucket(L_1,L_2,...,L_d)
```

4.2 RadixSort

- 假设我们要对每个有 d 个数字的 n 进制整数排序

```
1 RadixSort(A,d):
2   for i:=1 to d
3       use-bucket-sort-to-sort-A-on-digit-i
```